

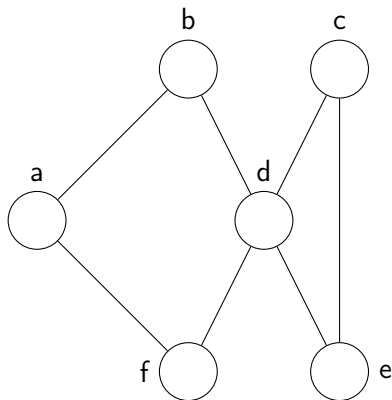
COMP1324 Algorithmics

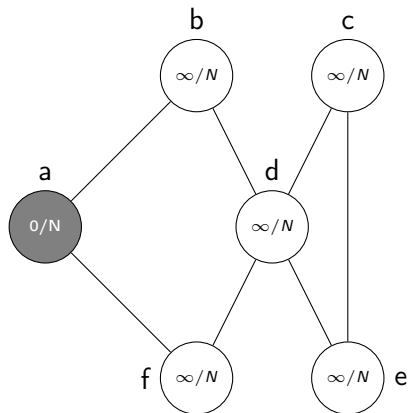
Revision 2025

Hikmat Farhat

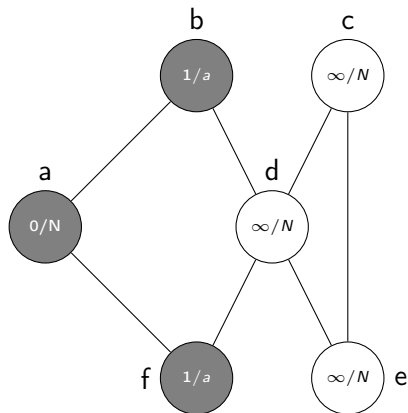
- Graphs: DFS, BFS, Kruskal, Prim, Bellman-Ford, Dijkstra
- Greedy, Dynamic programming
- Use simplex to solve LP problem.
- Bracktracking: enumerating solution space, complexity, branches
- Use DPLL
- Branch and bound

- Consider the graph below. Which of the following is a valid BFS traversal

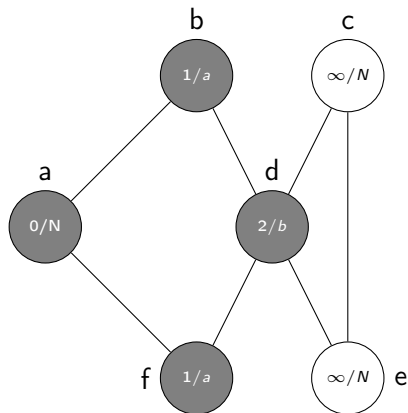




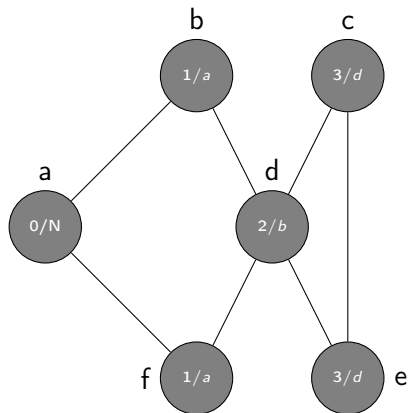
- ① a-b-c-d-e-f
- ② a-b-f-d-e-c
- ③ a-b-d-c-e-f
- ④ a-f-b-c-d-e



- ① a-b-c-d-e-f
- ② a-b-f-d-e-c
- ③ a-b-d-c-e-f
- ④ a-f-b-c-d-e



- ① a-b-c-d-e-f
- ② a-b-f-d-e-c
- ③ a-b-d-c-e-f
- ④ a-f-b-c-d-e



- 1 a-b-c-d-e-f
- 2 a-b-f-d-e-c
- 3 a-b-d-c-e-f
- 4 a-f-b-c-d-e

Shortest Path: Bellman-Ford Algorithm

Which of the following statements correctly identifies a key advantage of the **Bellman-Ford** algorithm compared to Dijkstra's algorithm?

- ☐ A It has a significantly faster worst-case time complexity of $O(V + E)$.
- ☐ B It is the only algorithm capable of finding the shortest path in a graph with positive cycles.
- ☐ C It can handle graphs with negative edge weights and detect the presence of negative cycles.
- ☐ D It uses a greedy approach to ensure each node is visited only once.

Shortest Path: Bellman-Ford Algorithm

Which of the following statements correctly identifies a key advantage of the **Bellman-Ford** algorithm compared to Dijkstra's algorithm?

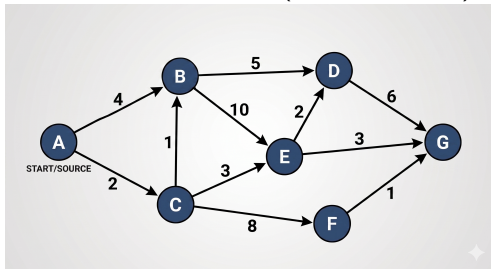
- ☐ A It has a significantly faster worst-case time complexity of $O(V + E)$.
- ☐ B It is the only algorithm capable of finding the shortest path in a graph with positive cycles.
- ☒ C It can handle graphs with negative edge weights and detect the presence of negative cycles.
- ☐ D It uses a greedy approach to ensure each node is visited only once.

Correct Answer: (C)

Dijkstra's algorithm may fail when edges have negative weights. Bellman-Ford relaxes all edges $|V| - 1$ times, allowing it to correctly compute distances with negative weights and identify negative cycles during a $|V|^{th}$ pass.

Dijkstra

Dijkstra's algorithm is used on the graph below. Select the correct order in which order the first 4 nodes removed from the queue and their shortest distance (in parentheses)



- ① A(0)-B(4)-C(2)-D(7)
- ② A(0)-C(2)-B(4)-D(7)
- ③ A(0)-C(2)-B(3)-E(5)
- ④ A(0)-C(2)-B(4)-E(5)

Graph Adjacencies

An undirected connected graph $G = (V, E)$ is represented by an adjacency matrix Adj of size $|V| \times |V|$ where $Adj[i][j] = 1$ iff $(i, j) \in E$.

What are the values of a and b in the following algorithm?

Algorithm 1: Graph Algorithm

input: An undirected connected graph $G = (V, E)$

Result: value of count

```
1 function GAlg(G)
2   foreach  $v \in V$  do
3     foreach  $i \in Adj[v][i]$  do
4        $a \leftarrow a + 1$ 
5       if  $Adj[v][i] = 1$  then
6          $b \leftarrow b + 1$ 
7   return  $a, b$ 
```

$a = |V|^2$ and $b = 2|E|$

Climbing stairs

- You are climbing a staircase. It takes n steps to reach the top.
- Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

function $F(n)$

```

/* initialise array of size n+1          */
A[1]=1
A[2]=2
for  $i \leftarrow 3$  to  $n$  do
|   A[i]=A[i-1]+A[i-2]
return A[n]

```

- Can reduce space complexity by keeping the last two values instead of array.

- You are given an array of prices of a stock.
- Design a DP algorithm to maximise your profit from buying on day i and selling on day j
- Example: $p = [7, 2, 6, 15, 9, 13, 19, 12, 7, 2]$ the max profit is 17, buying on day 1 and selling on day 6
- Hint: obtain a recursive expression for $M_j = \text{max profit when selling on day } j$



$$M_j = \max_{0 \leq i \leq j} P[j] - P[i]$$

$$\begin{aligned} M_{j+1} &= \max_{0 \leq i \leq j+1} P[j+1] - P[i] \\ &= \max(0, \max_{0 \leq i \leq j} P[j+1] - P[i]) \\ &= \max(0, \max_{0 \leq i \leq j} P[j+1] - P[j] + P[j] - P[i]) \\ &= \max(0, M_j + P[j+1] - P[j]) \end{aligned}$$

function Profit(P)

 /* allocate array of size n */

$M[0] \leftarrow 0$

for $j \leftarrow 1$ **to** $n - 1$ **do**

$M[i] \leftarrow \max(0, M[i - 1] + P[i] - P[i - 1])$

return $\max M$

Find the maximum of $x_1 + x_2$ with the following constraints:

$$2x_1 + x_2 \leq 12,$$

$$x_1 + 2x_2 \leq 9,$$

$$x_1, x_2 \geq 0.$$

- 1 7
- 2 10
- 3 15
- 4 23

The problem is already in standard form. Convert it to slack form by introducing slack variables x_3 and x_4 . Then the problem is put in tableau form as shown in the next slide

	x_1	x_2	x_3	x_4	RHS(b)
z	-1	-1	0	0	0
x_3	2	1	1	0	12
x_4	1	2	0	1	9

Entering variables: x_1 or x_2 . By Bland's rule we choose x_1 . Leaving variable is chosen by min ratio test: x_3 .

	x_1	x_2	x_3	x_4	RHS(b)
z	-1	-1	0	0	0
x_3	2	1	1	0	12
x_4	1	2	0	1	9

Pivot $x_1 \leftrightarrow x_3$

Solution

	x_1	x_2	x_3	x_4	RHS(b)
z	0	$-1/2$	$1/2$	0	6
x_1	1	$1/2$	$1/2$	0	6
x_4	0	$3/2$	$-1/2$	1	3

Entering variable x_2 . Ratios: 12 for x_1 , 2 for x_4 .

	x_1	x_2	x_3	x_4	RHS(b)
z	0	$-1/2$	$1/2$	0	6
x_1	1	$1/2$	$1/2$	0	6
x_4	0	$3/2$	$-1/2$	1	3

Pivot $x_2 \leftrightarrow x_4$

	x_1	x_2	x_3	x_4	RHS(b)
z	0	0	$1/3$	$1/3$	7
x_1	1	0	$2/3$	$-1/3$	5
x_2	0	1	$-1/3$	$2/3$	2

All coefficients in the z row are positive so we stop. Setting the non-basic variables to 0 we get the optimal values:

$$z = 7, x_1 = 5, x_2 = 2$$

Backtracking: Bounded Knapsack

Consider the **Bounded Knapsack Problem**, where an item can be used at most twice, with n items and a capacity C . A backtracking algorithm A is used to find the optimal solution. Which statements are true?

- ❶ A can stop searching once it finds a solution that fills the capacity C exactly.
- ❷ A visits the state-space tree in a depth-first manner.
- ❸ If there are multiple optimal solutions, A can find all of them.
- ❹ A visits at most $O(2^n)$ nodes.

Backtracking: Bounded Knapsack

Consider the **Bounded Knapsack Problem**, where an item can be used at most twice, with n items and a capacity C . A backtracking algorithm A is used to find the optimal solution. Which statements are true?

- ❶ A can stop searching once it finds a solution that fills the capacity C exactly. False
- ❷ A visits the state-space tree in a depth-first manner.
- ❸ If there are multiple optimal solutions, A can find all of them.
- ❹ A visits at most $O(2^n)$ nodes.

1. False: Filling the capacity doesn't guarantee the **maximum value**.

Backtracking: Bounded Knapsack

Consider the **Bounded Knapsack Problem**, where an item can be used at most twice, with n items and a capacity C . A backtracking algorithm A is used to find the optimal solution. Which statements are true?

- ❶ A can stop searching once it finds a solution that fills the capacity C exactly. False
- ❷ A visits the state-space tree in a depth-first manner. True
- ❸ If there are multiple optimal solutions, A can find all of them.
- ❹ A visits at most $O(2^n)$ nodes.

1. False: Filling the capacity doesn't guarantee the **maximum value**.

Backtracking: Bounded Knapsack

Consider the **Bounded Knapsack Problem**, where an item can be used at most twice, with n items and a capacity C . A backtracking algorithm A is used to find the optimal solution. Which statements are true?

- ① A can stop searching once it finds a solution that fills the capacity C exactly. False
- ② A visits the state-space tree in a depth-first manner. True
- ③ If there are multiple optimal solutions, A can find all of them. True
- ④ A visits at most $O(2^n)$ nodes.

1. False: Filling the capacity doesn't guarantee the **maximum value**.

Backtracking: Bounded Knapsack

Consider the **Bounded Knapsack Problem**, where an item can be used at most twice, with n items and a capacity C . A backtracking algorithm A is used to find the optimal solution. Which statements are true?

- ① A can stop searching once it finds a solution that fills the capacity C exactly. False
- ② A visits the state-space tree in a depth-first manner. True
- ③ If there are multiple optimal solutions, A can find all of them. True
- ④ A visits at most $O(2^n)$ nodes. False

- 1. False: Filling the capacity doesn't guarantee the **maximum value**.
- 4. False: Branching factor is 3 (0, 1, 2), so complexity is $O(3^n)$.

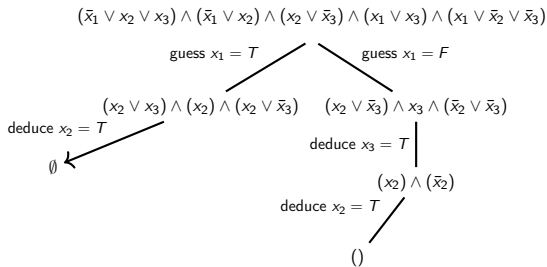
- Consider the Boolean formula. (Note: sometimes we use $\neg x$ and sometimes $\bar{x}$ whichever is more convenient)

$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- The value of x_1 is deduced
- DPLL might backtrack on x_2
- ϕ has more than one satisfying assignment.
- ϕ is unsatisfiable.

- At each step we try to reduce formula by using unit clause propagation then pure literal elimination.
- If the formula cannot be reduced further by these two rules then we try guessing the value of the remaining variables, smallest index first
- "might backtrack" on variable x means at least one possible assignment for x will lead to unsatisfiability.
- For example
$$(\bar{x}_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_1 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_3)$$
- DPLL will backtrack on x_1 even though $x_1 = T$ makes ϕ satisfiable but $x_1 = F$ makes it unsatisfiable.

Example



- Consider the Boolean formula. (Note: sometimes we use $\neg x$ and sometimes $\bar{x}$ whichever is more convenient)

$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

$$x_1 \text{ pure} \Rightarrow \phi = (x_2 \vee \bar{x}_3)$$

$$x_2 \text{ pure} \Rightarrow \phi = \emptyset$$

- The value of x_1 is deduced (T. Pure literal elimination)
- DPLL might backtrack on x_2 (F. x_2 is deduced pure lit.).
- ϕ has 3 sat. assignments (T)
- ϕ is unsatisfiable (F)

- An example of unsatisfiable formula.

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced
- 2 The value of x_3 is deduced
- 3 DPLL will backtrack on x_1
- 4 The formula is satisfiable

The DPLL search tree is on the next slide.

- An example of unsatisfiable formula.

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced **False**.
- 2 The value of x_3 is deduced
- 3 DPLL will backtrack on x_1
- 4 The formula is satisfiable

The DPLL search tree is on the next slide.

- An example of unsatisfiable formula.

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced **False**.
- 2 The value of x_3 is deduced **True**
- 3 DPLL will backtrack on x_1
- 4 The formula is satisfiable

The DPLL search tree is on the next slide.

- An example of unsatisfiable formula.

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced **False**.
- 2 The value of x_3 is deduced **True**
- 3 DPLL will backtrack on x_1 **True**
- 4 The formula is satisfiable

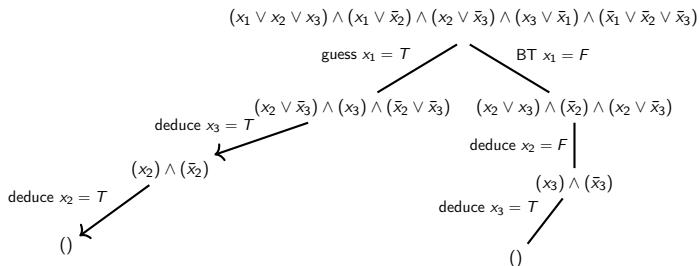
The DPLL search tree is on the next slide.

- An example of unsatisfiable formula.

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced **False**.
- 2 The value of x_3 is deduced **True**
- 3 DPLL will backtrack on x_1 **True**
- 4 The formula is satisfiable **False**

The DPLL search tree is on the next slide.



What is the primary purpose of the **Bounding** function in a Branch and Bound algorithm?

- Ⓐ To determine the order in which child nodes are generated.
- Ⓑ To estimate the best possible solution reachable from a specific node to decide if the branch should be explored.
- Ⓒ To provide a definitive solution for a sub-problem without further branching.
- Ⓓ To ensure that the algorithm follows a Depth-First Search (DFS) pattern.

What is the primary purpose of the **Bounding** function in a Branch and Bound algorithm?

- Ⓐ To determine the order in which child nodes are generated.
- Ⓑ To estimate the best possible solution reachable from a specific node to decide if the branch should be explored.
- Ⓒ To provide a definitive solution for a sub-problem without further branching.
- Ⓓ To ensure that the algorithm follows a Depth-First Search (DFS) pattern.

Correct Answer: (B)

The bounding function provides a limit that allows the algorithm to "prune" branches that cannot possibly contain an optimal solution.

Branch and Bound: Pruning Logic

In a **maximization** problem, under which condition is a node in the state-space tree pruned (discarded)?

- A When its calculated upper bound is less than or equal to the current best solution found so far.
- B When its calculated upper bound is greater than the current best solution.
- C When the node reaches a depth equal to half of the total number of items.
- D When the upper bound of the node is greater than the upper bound of its parent.

Branch and Bound: Pruning Logic

In a **maximization** problem, under which condition is a node in the state-space tree pruned (discarded)?

- Ⓐ When its calculated upper bound is less than or equal to the current best solution found so far.
- Ⓑ When its calculated upper bound is greater than the current best solution.
- Ⓒ When the node reaches a depth equal to half of the total number of items.
- Ⓓ When the upper bound of the node is greater than the upper bound of its parent.

Correct Answer: (A)

*In maximization, we track the best solution found so far. If a branch's **best possible** outcome (upper bound) cannot beat our current best, exploring it is a waste of time.*

Suppose that for each instance I of a maximisation problem the optimal value is $OPT(I)$. A certain approximation algorithm A is used to solve the problem. Let $A(I)$ be the value returned by A for instance I where $A(I) \geq OPT(I)/4$ for all I except for one instance, J , where $A(J) \geq OPT(J)/2$.

- 1 A is a $1/2$ -approximation algorithm
- 2 A is a $1/4$ -approximation algorithm

Factorial

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:
```

```
        return n * factorial(n - 1)
```

$$T(n) = T(n-1) + O(1) \Rightarrow T(n) = O(n)$$

$$\begin{aligned} T(n) &\leq T(n-1) + c \\ &\leq T(n-2) + 2c \\ &\leq T(n-3) + 3c \\ &\dots \\ &\leq T(0) + nc \\ &= O(n) \end{aligned}$$

```
def binary_search(arr, target, low, high):  
    if low > high:  
        return -1  
    mid = (low + high) // 2  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] > target:  
        return binary_search(arr, target, low, mid-1)  
    else:  
        return binary_search(arr, target, mid+1, high)
```

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = O(\log n)$$

$$\begin{aligned} T(n) &\leq T(n/2) + c \\ &\leq T(n/4) + 2c \\ &\leq T(n/8) + 3c \\ &\dots \\ &\leq T(n/2^k) + kc \\ &\dots \\ &\leq T(1) + kc \\ &\leq T(1) + c \log n \\ &= O(\log n) \end{aligned}$$

Exponentiation by Squaring

```
def power(x, n):  
    if n == 0:  
        return 1  
    half = power(x, n // 2)  
    return half * half if n % 2 == 0 else half * x
```

We consider only the special case when n is a power of 2. The general case gives the same result but it is more complicated.

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = O(\log n)$$

Merge Sort

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return merge(left, right)
```

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

$$\begin{aligned}
T(n) &\leq 2T(n/2) + cn \\
&\leq 2(2T(n/4) + cn/2) + cn \\
&\leq 2^2 T(n/2^2) + 2cn \\
&\leq 4(2T(n/8) + cn/4) + 2cn \\
&\leq 2^3 T(n/2^3) + 3cn \\
&\dots \\
&\leq 2^k T(n/2^k) + kcn \\
&\dots
\end{aligned}$$

For $2^k = n$ or $k = \log n$ we have

$$\begin{aligned}
T(n) &\leq nT(1) + ck \log n \\
&= O(n \log n)
\end{aligned}$$

Fibonacci (Naive)

```
def fib(n):  
    if n <= 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```

$$T(n) = T(n-1) + T(n-2) + O(1) \Rightarrow T(n) = O(2^n)$$

It is difficult to see a pattern when iterating the recurrence. We can use a tree to show the number of calls made. Instead we prove the claim by induction.

Fibonacci (Naive)

Base case:

For $n = 0$ and $n = 1$ the number of operations is 1. The claim holds.

Assume it is true up to $n - 1$. For n we have:

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + O(1) \\ &\leq 2^{n-1} + 2^{n-2} + c \\ &\leq 2^{n-1} + 2^{n-1} \\ &\leq 2^n \end{aligned}$$